CS3081 CLOUD COMPUTING - MINI PROJECT

Department of Computer Science & Engineering

Knowledge Distillation & Deployment of a Lightweight Command-Parsing Language Model

Student Details

Name: Dixant Soni
Roll No: 230103029
Course: CS3081 - Cloud Computing
GitHub: https://github.com/DIXANTOFFICIAL1/Knowledge-Distillation-and-Deployment-of-a-Lightweight-

Submission Details

Git Tag: `final`
Release: `v1.0.0  (model weights via GitHub Release assets - NOT committed to Git)`
AI Disclosure: See AI_USE.md - phi3.5 teacher model + Gemini assistant, all outputs verified

Abstract

This report presents the end-to-end design, implementation, and empirical validation of a cloud-native natural language command-parsing pipeline. We employ Sequence-Level Knowledge Distillation to compress phi-3.5-mini-instruct (3.8B parameters, Teacher) into EleutherAI/pythia-70m (70M parameters, Student) achieving 96.0% JSON validity and 70.0% full-frame accuracy. The student is packaged in a CPU-only Docker image (842 MB, below 2 GB limit), deployed to a local k3d Kubernetes cluster with SeaweedFS S3 storage, Traefik Ingress, HPA, and zero-downtime rolling updates. A 5-stage CI/CD pipeline enforces 95% validity quality gates and secret scanning. Serving cost: 0.37 INR per 1,000 requests.

Report Contents

Task 1: Object Storage & Parallel Teacher Labelling **pg 2**
Task 2: Sequence-Level Knowledge Distillation & Evaluation **pg 3-4**
Task 3: Inference Microservice in a Container **pg 5**
Task 4: Kubernetes Deployment, Scaling & Reliability **pg 6**
Task 5: Subnetting, VLSM & Stateless NACL Design **pg 7**
Task 6: CI/CD, Reproducibility & Testing **pg 8**
Task 7: Performance, Cost Estimation & SLO **pg 8**

Task 1 Object Storage and Parallel Teacher Labelling

1.1 Cluster & Storage Setup

A local Kubernetes cluster was provisioned using k3d (cluster/k3d.yaml) with a single server node and agent node. The loadbalancer was configured to forward host port 8080 to ingress port 80, exposing Traefik for external traffic. SeaweedFS was deployed into namespace data via k8s/seaweedfs.yaml, running as a combined master-volume-S3 server on port 8333. Two S3 buckets were created: datasets (for input data) and models (for trained weights). The raw dataset commands_pool.jsonl (3,000 SLURP commands, CC BY 4.0) was uploaded to s3://datasets/commands_pool.jsonl using the SeaweedFS S3 API.

1.2 Teacher Model & Prompt Engineering

phi3.5-mini-instruct (3.8B parameters) was served locally via Ollama with deterministic settings: temperature=0.0, seed=42, ensuring identical outputs on reruns. The teacher prompt (prompts/teacher_prompt.txt) constrains the model to output a single valid JSON object with exactly four keys: intent, date, time, and place. The prompt enumerates all 8 valid intents and states three strict slot extraction rules: (a) copy value verbatim from the command, (b) extract the shortest meaningful phrase without prepositions, (c) set null when the slot is absent.

1.3 Parallel Sharded Worker Architecture

To overcome the throughput bottleneck of single-threaded CPU inference (~2.2 seconds per command), two worker pods were deployed concurrently in namespace pipeline. Each worker reads SHARD_INDEX and SHARD_COUNT environment variables and applies the deterministic modulo partition: a command at zero-based index i is processed exclusively by worker SHARD_INDEX = i % SHARD_COUNT. This guarantees zero overlap and zero gaps across the full 3,000-command pool, independent of execution order.

Worker 0: processes commands 0, 2, 4, ... (even indices) ? uploads to s3://datasets/teacher/shard-0.jsonl

Worker 1: processes commands 1, 3, 5, ... (odd indices) ? uploads to s3://datasets/teacher/shard-1.jsonl

1.4 Data Cleaning Pipeline (worker/clean.py)

The two shard files were merged and filtered by worker/clean.py applying five sequential validation rules:

(a) JSON Validity: Reply must parse as a valid JSON object (json.loads). Rejects malformed or truncated responses.

(b) Key Completeness: Parsed object must have exactly the keys {intent, date, time, place} - no more, no fewer.

(c) Intent Validity: intent value must be one of the 8 enumerated intents in schema.json.

(d) Slot Substring Check: Non-null slot values must appear verbatim as a lowercase substring of the command text.

(e) Duplicate Deduplication: Commands with identical text appear only once; the entry with the lexicographically smallest id is retained.

Filter Step	Description	Impact	Count
Input Records	Raw lines from commands_pool.jsonl	Baseline	3,000
Rejected (b) Wrong Keys	Missing or extra keys in parsed object	Dropped	18
Rejected (d) Slot Halluc.	Slot value not substring of command	Dropped	65
Records Kept (a+b+c+d+e+kept = 3,000 ?)		**2,800**	

The cleaned dataset (2,800 labelled records, sorted by id) was written to datasets/v1/train.jsonl and its SHA-256 checksum saved to datasets/v1/train.sha256. The first 1,500 records were used for student training (Task 2).

Task 2 Training the Student - Sequence-Level Knowledge Distillation

2.1 Distillation Methodology

Sequence-Level Knowledge Distillation (Kim & Rush, EMNLP 2016) trains the student model to reproduce the teacher's argmax output sequences rather than its internal probability distributions. The student minimises standard cross-entropy loss against the teacher's discrete token outputs, treating them as ground-truth supervision signals:

Each training example is formatted using the prompt template that mirrors the inference-time format, ensuring the student learns the conditional mapping directly:

2.2 Why Logit-Based Distillation is Not Applicable Here

Logit-based distillation (Hinton et al., 2015) minimises KL-divergence between the student's and teacher's full output distributions over their shared vocabulary. This requires identical tokenisers and vocabulary sizes. phi3.5 uses a SentencePiece BPE tokeniser with vocabulary size 32,064, while pythia-70m uses GPT-NeoX byte-level BPE with vocabulary size 50,304. Because token indices map to entirely different subword units, aligning logit vectors is undefined without introducing a learned cross-vocabulary projection layer - an engineering complexity that is out of scope for a 70M CPU-only student. Additionally, logit distillation requires running the teacher in inference mode with the student's partial generations at each step, which is computationally infeasible when the teacher is served over an HTTP API.

2.3 Hyperparameter Configuration (training/config.yaml)

Parameter	Value	Design Rationale
Base Model	EleutherAI/pythia-70m	Smallest Pythia variant with full HF support; 70M params
Learning Rate	5e-5 (AdamW)	Conservative LR prevents catastrophic forgetting on the small student
Max Seq Length	128 tokens	Covers all command + JSON target pairs; truncation rate < 1%
Training Epochs v2	4 epochs	One additional epoch for rolling-update deployment demonstration
		onl
torch / transformers	2.2.2 / 4.38.2	Pinned in requirements.txt for strict environment reproducibility

2.4 Training Loss Trajectory

Training was executed on CPU (make train). The loss profile confirms healthy convergence - the model first acquires JSON structural syntax in Epoch 1, then stabilises intent classification in Epoch 2, before refining slot boundary extraction in Epoch 3:

Epoch	Loss (CE)	Relative Drop	Interpretation
1	2.4512	Baseline	Learns JSON schema: braces, colon, quotes, null keyword
			cabulary to 8 in
3	0.9854	-39.3%	Slot extraction refines; date/time/place phrase boundaries tighte

The smoke test make train SUBSET=50 completes in under 90 seconds on CPU, allowing examiners to validate the full training pipeline without requiring hours of execution.

2.5 Model Storage & Versioning

Trained model weights (pytorch_model.bin), tokenizer (tokenizer.json, vocab.json), and config (config.json) for both v1 and v2 were uploaded to SeaweedFS under s3://models/v1/ and s3://models/v2/ respectively. Files are NOT committed to the Git repository. Instead, they are attached as GitHub Release assets under tag v1.0.0, satisfying the asset-separation requirement. A directory-level SHA-256 checksum is stored in models/v1/model.sha256 for integrity verification.

Task 2 (cont.) Three-Model Evaluation, Comparative Analysis & Limitations

2.6 Evaluation Protocol

All three model variants were evaluated on data/validation.jsonl - 100 manually labelled, never-trained-on SLURP commands. Predictions were generated by the respective inference code and formatted as JSONL with fields id and prediction (the raw model output string). tools/score.py parsed each prediction, validated it against schema.json, and computed six metrics: json_validity, intent_accuracy, date_accuracy, time_accuracy, place_accuracy, and full_frame (all four fields simultaneously correct).

Model Variant	JSON Valid	Intent	Date	Time	Place	Full Frame
Untrained Pythia-70M	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
Distilled Pythia-70M (Student)	**96.00%**	**96.00%**	**76.00%**	**84.00%**	**96.00%**	**70.00%**

Rubric Compliance:
Student JSON Validity 96% >= required 95% | Student Full Frame 70% >= required 41% (50% of teacher 82%) PASS

2.7 Error Analysis & Per-Slot Discussion

JSON Validity (96% vs 0% baseline): Post-distillation, the student reliably generates syntactically valid JSON objects, demonstrating that causal language-model fine-tuning on structured targets effectively internalises schema constraints. The 4 invalid outputs were cases where the model generated a valid JSON followed by an extra natural-language sentence.

Intent Accuracy (96%): The student matches the teacher’s intent classification, confirming that 8-class command intent is a learnable pattern well within 70M parameters’ capacity. Intent tokens are high-frequency and appear early in every output sequence, making them easy to reinforce.

Date Accuracy (76%) - lowest slot: Date expressions are highly context-dependent multi-word spans (e.g., ’the week after next’, ’the month of may’, ’this coming friday’). The student’s greedy decoding strategy, combined with its small context window capacity, causes it to truncate or misidentify span boundaries for complex temporal expressions more than the teacher.

Time Accuracy (84%): Better than date but lower than place. Time expressions containing number-word hybrids (e.g., ’seven thirty’, ’two hundred’) occasionally cause boundary confusions, though simple forms (’six am’, ’five pm’) are reproduced accurately.

Place Accuracy (96%): City and location names are lexically distinctive, typically capitalised proper nouns infrequent in pre-training data, making them easy to copy verbatim from the input command.

2.8 One Real Limitation of Sequence-Level Distillation

Limitation: Teacher Bias Entrenchment via Hard-Target Supervision
Because sequence distillation trains on the teacher’s argmax token sequence only, the student receives zero signal regarding the teacher’s uncertainty over alternative plausible outputs. If phi3.5 produces a slot hallucination with 51% confidence (e.g., ’paris’ for a command that never mentions a city), the student learns this error as 100% ground truth, amplifying and entrenching the teacher’s systematic biases. Logit-based KD would convey the teacher’s uncertainty but is infeasible here.

Task 3 The Inference Microservice in a Container

3.1 Service Architecture (service/app.py)

The inference service is implemented using FastAPI and Uvicorn, binding to 0.0.0.0:8000. It exposes three endpoints with distinct responsibilities:

Method	Endpoint	Behaviour	Specification
POST	/parse	Accepts {text: ...} JSON body. Calls model. Validates output against schema.json	Returns 200 + 4-key JSON | 500 on schema fail
GET	/readyz	Readiness probe. Returns 503 during S3 download/model loading	{status: loading} 200 {status: ready}

3.2 Configuration via Environment Variables

- MODEL_URI: S3 prefix of model artefacts (e.g. models/v1). Never hardcoded.
- MODEL_VERSION: String written to X-Model-Version response header. Enables version tracing in rolling updates.
- S3_ENDPOINT: Full URL of SeaweedFS S3 gateway. No default; must be injected by Kubernetes.
- MAX_NEW_TOKENS: Maximum generation length. Defaults to 64 (sufficient for all JSON outputs).
- FAULT_INJECT: Set to 1 to deliberately return ’not json’, forcing HTTP 500 for quality-gate testing.
- AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY: Injected exclusively from k8s/secret.yaml via secretKeyRef, never stored in environment files or Dockerfiles.

3.3 Startup Sequence & Probe Coordination

On container startup, the lifespan context manager immediately marks the pod as alive (/healthz = 200) but not ready (/readyz = 503). It then connects to SeaweedFS, paginates the S3 bucket under MODEL_URI, downloads all model files to /app/model, and loads them into memory via Hugging Face AutoModelForCausalLM. Only after successful model.eval() does it flip state[ready] = True, enabling /readyz to return 200 and allowing Traefik Ingress to route traffic to the pod. This prevents any 502 Bad Gateway errors during cold starts or rolling updates.

3.4 Schema Validation & Fault Injection

Every model-generated string is passed through validate_schema() before being returned as HTTP 200. The validator confirms: (a) the string parses as valid JSON, (b) the parsed object contains exactly the keys {intent, date, time, place}, (c) intent is one of the 8 enumerated values, and (d) slot values are either null or strings. Any validation failure causes a HTTP 500 response with body {error: invalid_model_output}. When FAULT_INJECT=1, the generated text is overwritten with ’not json’ before validation, guaranteeing a 500 response - used in CI quality-gate failure demonstrations.

3.5 Dockerfile & Image Size Optimisation

The container is built from python:3.10-slim, chosen for its minimal surface area. The critical optimisation is installing the CPU-only PyTorch wheel from the dedicated index (--extra-index-url https://download.pytorch.org/whl/cpu), which excludes CUDA runtime libraries. The standard torch+cu121 wheel is ~2.5 GB; the CPU-only wheel is ~200 MB, enabling the final image to fit within the 2 GB limit. No model weights are COPY’d into the image - they are downloaded at runtime from SeaweedFS.

Measured Evidence: Container Benchmark (1 CPU, 1 GiB Memory Limit)

```
Command:  docker run --cpus=1.0 --memory=1g -p 8000:8000 parser-service:latest
Workload: 50 sequential POST /parse requests
```

docker stats output: CPU 42.8% | MEM 486.2 MiB / 1.0 GiB (47.48%) | OOM Kills: 0

Result: 50/50 requests returned HTTP 200. Image size: 842 MB (< 2 GB requirement PASS)

The measured peak memory of 486 MiB (47.5% of quota) leaves a comfortable headroom, confirming the CPU-only PyTorch architecture is viable for memory-constrained inference workloads. Full log preserved in results/docker_stats_peak_memory.txt.

Task 4 Kubernetes Deployment, Scaling and Reliability

4.1 Deployment Architecture

The inference service was deployed to namespace app via k8s/deployment.yaml with 2 initial replicas. Each pod carries CPU request 250m and limit 1000m; memory request 512Mi and limit 1Gi. Liveness (/healthz, period 5s, failure threshold 3) and readiness (/readyz, period 3s, initial delay 5s) probes are configured. The ClusterIP service (k8s/service.yaml) forwards internal port 8000. Traefik Ingress (k8s/ingress.yaml) routes /parse, /healthz and /readyz from host port 8080 to the service. HPA (k8s/hpa.yaml) manages 1-2 replicas targeting 70% average CPU utilisation.

4.2 Empirical Reliability Experiments

Four cluster experiments were conducted, each under a live Locust load test (4 users, http://localhost:8080). Logs are preserved in results/. Local single-node limitation acknowledged: experiments demonstrate logical Kubernetes behaviour, not physical multi-machine rack-level fault tolerance.

Experiment	Procedure	Measured Evidence	Compliance
Rolling Update	kubectl set env ... MODEL_VERSION=v2 during load test, then kubectl rollout undo	0 failures / 260 requests (0.0% error rate). Headers switched v1->v2->v1.	PASS (<= 0.5%)
Cold Start	Delete all pods; time pod creation to /readyz 200. Repeat x3.	Run 1: 4.82s Run 2: 4.31s Run 3: 4.55s Median: 4.55s. download logged.	Evidence logged
			0%.
HPA Scale-Up	Locust load pushed observed CPU to 78% (above 70% target threshold)	HPA scaled from 1 to 2 replicas within 45 s. Post-load cooldown to 1 replica.	PASS

4.3 Rolling Update Detail

The deployment strategy is RollingUpdate with maxSurge=1 and maxUnavailable=0, guaranteeing that at least 2 pods are always Ready during the transition. During the v2 rollout: a third pod was scheduled and waited for /readyz 200 before the first v1 pod entered Terminating. Traefik's endpoint controller observed readiness withdrawal and stopped routing to the terminating pod within one poll interval (~2s), producing zero failed requests across the transition.

4.4 Known Limitation

Improvement: Deploy to a multi-node cloud-managed cluster (e.g. GKE Autopilot) across multiple availability zones with PodAntiAffinity rules to distribute replicas across separate nodes, enabling genuine hardware-level fault tolerance and geographic failover.

Task 5 Subnetting (VLSM) and Network Access Control List Design

5.1 VLSM Subnet Allocation for 10.20.0.0/16

The /16 address block (10.20.0.0 - 10.20.255.255, 65,536 total addresses) is subnetted using Variable Length Subnet Masking (VLSM). Allocation proceeds from the largest host requirement to smallest, placed contiguously from 10.20.0.0 to minimise wasted address space. The host formula is Usable = 2^h - 2 where h = host bit count:

Subnet	Need	Network Addr	Prefix	Usable Host Range	Broadcast	Usable
app	1,000	10.20.0.0	/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022
data	100	10.20.5.0	/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126

VLSM Bit-width Derivations:

```
app:         2^10 - 2 = 1022 >= 1000  =>  /22   Block: 10.20.0.0 to 10.20.3.255  (1024 addrs)
public:      2^8  - 2 = 254  >= 250   =>  /24   Block: 10.20.4.0 to 10.20.4.255  (256 addrs)
data:        2^7  - 2 = 126  >= 100   =>  /25   Block: 10.20.5.0 to 10.20.5.127  (128 addrs)
```

5.2 Stateless NACL Rules for the Public Subnet (network/nacl.json)

Rule	Dir.	Protocol	Port Range	CIDR	Action	Purpose
100	IN	ALL	Any	198.51.100.0/24	DENY	Block abusive /24 before any allow rules (rule-ordering
210	IN	TCP	443	0.0.0.0/0	ALLOW	Public HTTPS inbound to load balancer
400	IN	TCP	1024-65535	10.20.0.0/22	ALLOW	Return traffic (ephemeral ports) from app subnet to loa
						e pods
200	OUT	TCP	1024-65535	0.0.0.0/0	ALLOW	Return packets to external clients on ephemeral ports

5.3 Why Stateless NACLs Require Explicit Ephemeral-Port Return Rules

Unlike stateful firewalls (e.g. AWS Security Groups, iptables conntrack), a Network ACL is completely stateless - it maintains no connection state table. Each packet is inspected in isolation. When a client OS sends a TCP SYN to port 443, the inbound Rule 210 permits it. However, the server's HTTP response travels as a separate outbound packet to the client's dynamically chosen ephemeral source port (chosen by the OS in the range 1024-65535 from RFC 6056). Without outbound Rule 200 explicitly permitting TCP to destination ports 1024-65535 to 0.0.0.0/0, every response packet is silently dropped at the NACL, making bidirectional TCP impossible despite the inbound allow rule. The same logic applies to inbound return traffic: application pod responses to the load balancer arrive on the LB's ephemeral source port, requiring inbound Rule 400.

5.4 Why Rule Ordering is Critical for 198.51.100.0/24

NACL rules are evaluated in strict ascending numerical order; the first matching rule terminates evaluation (first-match wins). The abusive CIDR 198.51.100.0/24 is a strict subset of 0.0.0.0/0. If the HTTP allow rules (Rules 200/210 for 0.0.0.0/0) were assigned a lower rule number than the deny rule, an HTTP request from 198.51.100.9 would match the allow rule first and be passed. Placing the deny at Rule 100 (before 200/210) ensures all traffic from that prefix is unconditionally dropped before any permissive rule is ever reached.

Score: 6 / 6 (100% OK - all 6 test packets matched expected decisions)

Task 6 Reproducibility, Testing, and CI/CD Pipeline

6.1 Makefile Automation

make up: Creates the k3d cluster, builds the Docker image, imports it into k3d (no registry push required), applies SeaweedFS, secrets, deployment, service, ingress, and HPA in <= 3 compound commands.

make down: Fully deletes the k3d cluster (k3d cluster delete cloud-mini-cluster) and cleans build artefacts.

make train SUBSET=50: Runs a 50-sample smoke test on CPU in < 90s, allowing examiners to validate the full pipeline.

make test: Runs pytest unit tests AND python tools/nacl_sim.py --check in a single command.

6.2 GitHub Actions CI/CD Pipeline (.github/workflows/ci.yml)

Stage	Tool/Command	Gate Requirement	Result
1 Linting	flake8 service/ worker/ training/	Zero E9/F63/F7/F82 syntax errors	**PASSED**
			PASSED
3 Evaluation	gh release download v1.0.0; score.py	Run inference on validation.jsonl from Release asset	**PASSED**
			PASSED 96%
5 Secret Scan	gitleaks/gitleaks-action@v2	Zero credential leaks in entire repository history	**0 Leaks**

Quality Gate Failure Demonstration: The pipeline was intentionally triggered with the untrained pythia-70m (json_validity=0%). score.py exited with status 1, failing the GitHub Actions workflow run. Evidence preserved at: /actions/runs/11823901452 on the GitHub repository.

Task 7 Performance Measurement, Cost Estimation and SLO

7.1 Locust Load Test Results (4 users, 2 minutes)

Metric	Measured Value	Requirement	Status	Detail
p50 Latency	110 ms	< 5,000 ms	**PASS**	Median across 720 requests
			PASS	95th-percentile; no request > 420ms
Throughput	6.00 req/s	-	-	720 total requests in 120 s
			PASS	0 failures / 720 requests
CPU (kubectl top)	42.8% / 1 core	< 1000m limit	**PASS**	Headroom of 57.2% available
			PASS	47.5% memory quota used

7.2 Cost Estimation (rate_card.json)

NR/hr

INR

7.3 SLO & Error Budget

SLO: 99% of POST /parse requests return HTTP 200 within 3,000ms over any 2-minute observation window. Error budget for a 720-request window: 720 x (1 - 0.99) = 7.2 allowed failures. Actual failures: 0 - budget consumed: 0% (7.2 requests remaining). The service is well within SLO.

7.4 Comprehensive Review Table

Concern	Measured Evidence	Limitation Found	Proposed Improvement
Data & Storage	2,800 records kept; 842 MB image; SHA-256 verified	S3 latency adds ~1.5s to cold start	PersistentVolumeClaim with pre-cached weights
			r
Latency & Throughput	p50:110ms; p95:260ms; 6 req/s	CPU-only autoregressive decoding bottleneck	ONNX Runtime INT8 quantization -> sub-50ms
			eline
Reliability	0% fail on update/recovery; 4.55s to [illegible]	Single physical node - no rack-level HA	Multi-zone GKE with PodAntiAffinity rules
			here